



Data Structures & Algorithms

Title: Student Marks Management System using Data Structures

The team for this project consists of the following members:

- 1. Mohammed Kamal**
- 2. Dina Mohammed**
- 3. Dyako Abubakr**
- 4. Zhir Daniel**

Student Marks Management System

Contents

Introduction	3
Problem Statement.....	3
Proposed Solution.....	5
Design	5
Why Use a Hash Table?	5
Why Not Use Other Data Structures?	6
Arrays	6
Linked Lists	6
Binary Search Trees (BSTs).....	6
Stacks.....	7
Queues.....	7
Flexibility of Hash Table	8
Dynamic Handling and Collision Resolution.....	8
Implementation	9
Conclusion.....	13

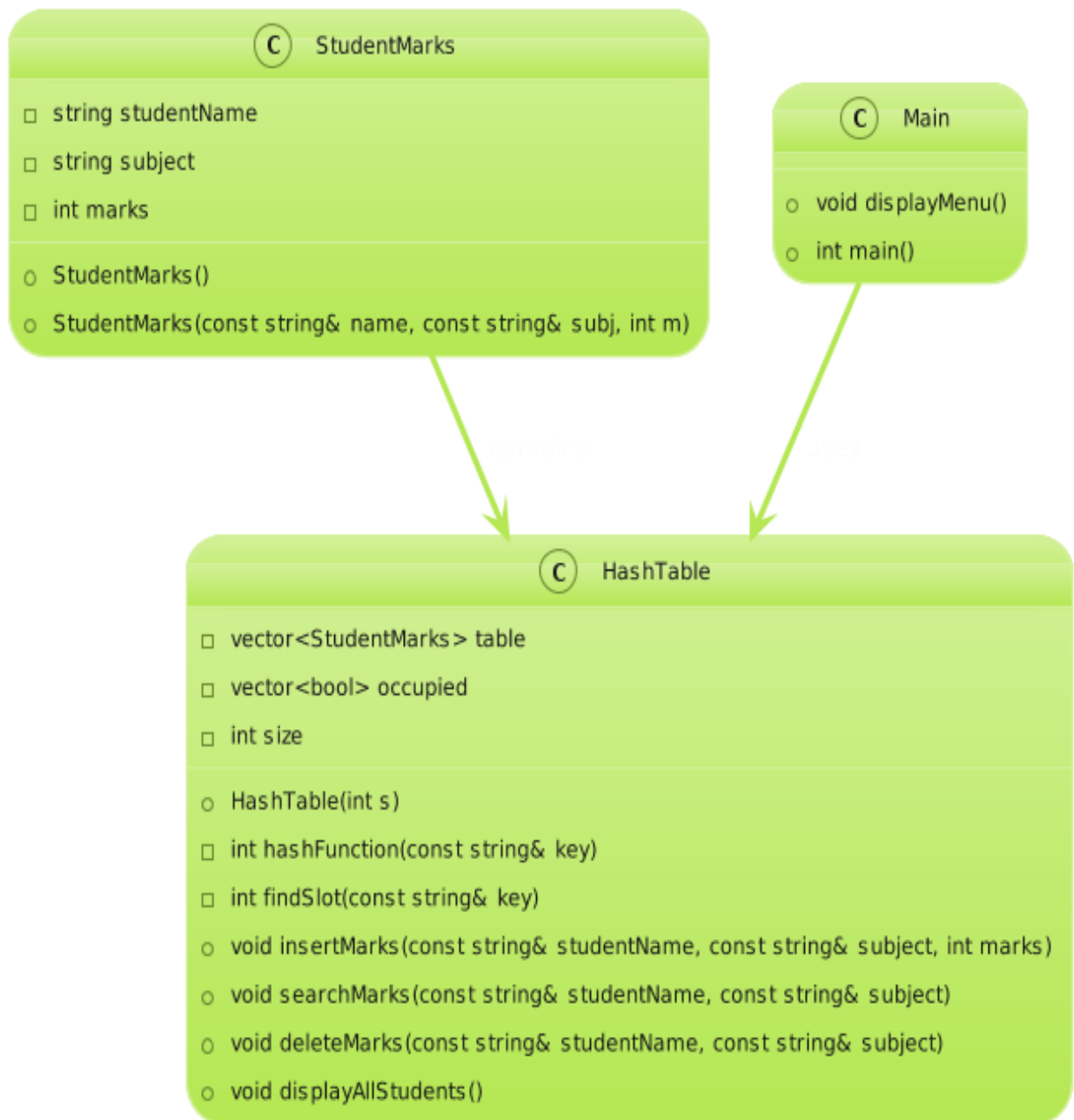
Introduction

The Student Marks Management System is a critical tool in educational institutions, designed to efficiently manage and retrieve students' academic records. This system aims to store, update, and delete student marks for different subjects, ensuring that the data is organized and accessible for various administrative and academic purposes. By leveraging modern data structures and algorithms, the system ensures that operations such as insertion, search, and deletion are performed swiftly and accurately, enhancing the overall efficiency and reliability of managing student data.

Problem Statement

In educational institutions, managing student marks across multiple subjects can be a daunting task. Traditional paper-based methods are prone to errors, time-consuming, and inefficient. Even basic digital systems that use simple arrays or lists face challenges in scalability, speed, and data integrity. The main problems include:

1. **Data Insertion Issues:** Inserting new student records or updating existing ones in large datasets can be slow and cumbersome with basic data structures.
2. **Search Inefficiency:** Quickly retrieving a student's marks for a particular subject is crucial, but linear search methods in simple lists or arrays are inefficient.
3. **Deletion Complexity:** Removing a student's records while maintaining the integrity and order of the dataset is complex with basic structures.
4. **Collision Handling:** In systems that use hash-based structures without proper collision management, data integrity issues can arise, leading to inaccurate records.



Proposed Solution

To address these issues, we propose implementing a Student Marks Management System using a hash table data structure. Hash tables provide efficient data retrieval, insertion, and deletion operations by utilizing a hash function to map keys (in this case, student names) to specific indices in an array. This method ensures constant time complexity ($O(1)$) on average for these operations, significantly improving performance over other data structures.

Insert: Add marks for a student in a particular subject.

Search: Retrieve the marks of a student for a specific subject.

Delete: Remove the marks of a student for a particular subject.

Design

The design of the Student Marks Management System is centered around the use of a hash table, which offers several advantages over other data structures. The hash table will manage the relationships between student names, subjects, and marks efficiently. Here's why a hash table is chosen and how it compares to other data structures.

Why Use a Hash Table?

A hash table is chosen for several compelling reasons:

1. **Efficient Data Retrieval:** Hash tables provide $O(1)$ average time complexity for search, insertion, and deletion operations. This is crucial for a system that requires frequent and fast access to student records.
2. **Dynamic Size Management:** Although arrays used within hash tables have a fixed initial size, the table can be dynamically resized to maintain efficiency as the number of elements grows. This adaptability ensures that the system can handle varying amounts of student data without significant performance degradation.
3. **Collision Handling:** Hash tables can implement effective collision handling mechanisms, such as linear probing, quadratic probing, or chaining. These techniques ensure that even when multiple keys hash to the same index, the data integrity and access speed are maintained.
4. **Memory Utilization:** Hash tables allocate memory as needed and avoid the need for large contiguous memory blocks, unlike arrays. This results in better memory

utilization and management, particularly important for systems handling large datasets.

Why Not Use Other Data Structures?

Arrays

Fixed Size: Arrays require knowing the maximum number of elements beforehand, making them less suitable for applications with dynamic data sizes. This limitation can lead to inefficient memory use or the need for frequent resizing.

Inefficient Insertions/Deletions: Operations like insertion and deletion require shifting elements, resulting in $O(n)$ time complexity, which is not ideal for large datasets.

Linked Lists

Search Inefficiency: While linked lists provide $O(1)$ time complexity for insertion and deletion when the position is known, searching for an element takes $O(n)$ time on average. This inefficiency makes linked lists less suitable for applications requiring fast data retrieval.

Memory Overhead: Linked lists use additional memory for storing pointers, which can increase the overall memory usage compared to hash tables.

Binary Search Trees (BSTs)

Balancing Complexity: Maintaining a balanced tree can be complex, and the performance can degrade to $O(n)$ in the worst case for unbalanced trees. Although self-balancing trees like AVL or Red-Black trees exist, their implementation is more complex compared to hash tables.

Higher Average Time Complexity: Even in balanced trees, the average time complexity for operations is $O(\log n)$, which is higher than the $O(1)$ average time complexity of hash tables.

Stacks

LIFO Structure: Stacks operate on a Last In, First Out (LIFO) principle, which is not suitable for managing student records where arbitrary access and modification are required.

Limited Use Case: Stacks are more suited for recursive algorithms and certain specific use cases like parsing expressions, rather than general data storage and retrieval.

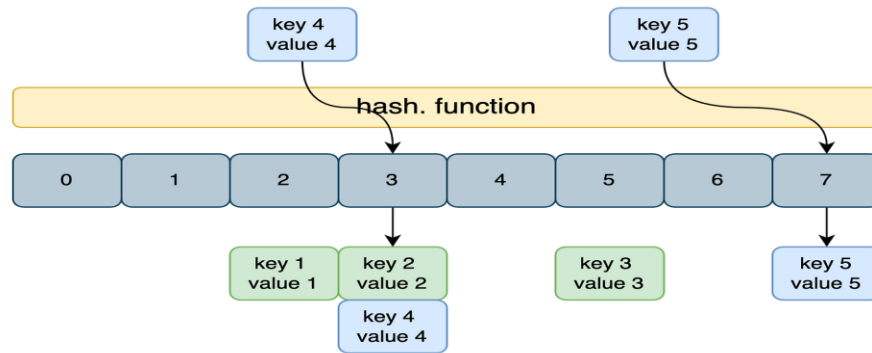
Queues

FIFO Structure: Queues operate on a First In, First Out (FIFO) principle, which does not align with the requirements of managing student records where any record might need to be accessed at any time.

Prioritization Not Required: Priority queues are designed for scenarios where elements need to be processed based on priority. Managing student marks does not require such prioritization, making queues less suitable.

Flexibility of Hash Table

The hash table manages relationships between student names, subjects, and marks efficiently by providing constant time complexity for the core operations. Each entry in the hash table represents a student-subject-mark relationship, facilitating easy access, insertion, and deletion of student data.



Dynamic Handling and Collision Resolution

Dynamic Insertion and Deletion: The hash table allows for dynamic insertion and deletion of entries. This flexibility is crucial for handling varying amounts of student data efficiently.

Collision Resolution: The use of linear probing for collision handling ensures that even when multiple keys hash to the same index, the data integrity is maintained. Linear probing involves finding the next available slot in the array, ensuring that all data is stored without loss.

Implementation

Initialization and Structure Definition

The implementation starts with the definition of the `StudentMarks` structure, which holds the details of a student's marks for a specific subject. This structure includes a default constructor and a parameterized constructor for initializing the attributes `studentName`, `subject`, and `marks`.

```
// Structure to hold the student's marks
struct StudentMarks {
    string studentName;
    string subject;
    int marks;

    StudentMarks() : studentName(""), subject(""), marks(0) {}

    StudentMarks(const string& name, const string& subj, int m) :
        studentName(name), subject(subj), marks(m) {}
};
```

Hash Table Class Definition

The `HashTable` class is the core component of the system, managing the storage and retrieval of student marks. The class contains several private members and methods, as well as public methods to perform various operations efficiently.

```
// Custom Hash Table class
class HashTable {
private:
    vector<StudentMarks> table;
    vector<bool> occupied;
    int size;

    // Hash function to calculate hash value of a key
    int hashFunction(const string& key) {
        int hash = 0;
        for (char c : key) {
            hash = (hash * 31 + c) % size;
        }
        return hash;
    }

    // Function to handle collisions using linear probing
    int findSlot(const string& key) {
        int index = hashFunction(key);
        while (occupied[index] && table[index].studentName != key) {
            index = (index + 1) % size;
        }
        return index;
    }

public:
    HashTable(int s) : size(s), table(s), occupied(s, false) {}
};
```

Insertion

The `insertMarks` method is responsible for adding a new student record or updating an existing one. This method calculates the appropriate slot for the student using the `hashFunction` and `findSlot` methods. If the slot is already occupied by the same student, the record is updated; otherwise, a new entry is created.

```
// Function to insert or update marks for a student
// Insertion:
void insertMarks(const string& studentName, const string& subject, int marks) {
    int index = findSlot(studentName);
    table[index] = StudentMarks(studentName, subject, marks);
    occupied[index] = true;
    cout << "Inserted/Updated marks for " << studentName << " in " << subject << endl;
}
```

Searching

The `searchMarks` method looks for a specific student's marks in a given subject. It uses the `findSlot` method to locate the student record. If the record is found and the subject matches, the method displays the marks; if not, it indicates that no record was found.

```
// Function to search and display marks for a student
// Searching:
void searchMarks(const string& studentName, const string& subject) {
    int index = findSlot(studentName);
    if (occupied[index] && table[index].subject == subject) {
        cout << studentName << " has " << table[index].marks << " marks in " << subject << endl;
    } else {
        cout << "No marks found for " << studentName << " in " << subject << endl;
    }
}
```

Deletion

The `deleteMarks` method removes the student record that matches the specified student name and subject. It uses the `findSlot` method to locate the record and then marks the slot as unoccupied if the record is found and the subject matches.

```
// Function to delete marks for a student
// Deletion:
void deleteMarks(const string& studentName, const string& subject) {
    int index = findSlot(studentName);
    if (occupied[index] && table[index].subject == subject) {
        occupied[index] = false;
        cout << "Deleted marks for " << studentName << " in " << subject << endl;
    } else {
        cout << "No marks found for " << studentName << " in " << subject << endl;
    }
}
```

Display

The `displayAllStudents` method traverses the hash table and prints the details of each student record. It iterates through the `table` and displays the details of all occupied slots, providing a comprehensive view of all stored student records.

```
// Function to display all student details in a table
// Display:
void displayAllStudents() {
    cout << left << setw(20) << "Student Name" << setw(20)
    << "Subject" << setw(10) << "Marks" << endl;

    cout << "-----" << endl;
    for (int i = 0; i < size; ++i) {
        if (occupied[i]) {
            cout << left << setw(20) << table[i].studentName
            << setw(20) << table[i].subject
            << setw(10) << table[i].marks << endl;
        }
    }
};

// Function to display the menu
void displayMenu() {
    cout << "\nStudent Marks Management System\n";
    cout << "1. Insert/Update Marks\n";
    cout << "2. Search Marks\n";
    cout << "3. Delete Marks\n";
    cout << "4. Display All Student Details\n";
    cout << "5. Exit\n";
    cout << "Enter your choice: ";
}
```

Main Function

The main function provides a menu-driven interface for the user to interact with the system. It continuously displays a menu and processes user input, allowing the user to insert, search, delete, and display student marks. Depending on the user's choice, it calls the appropriate method in the `HashTable` class to perform the desired operation.

```
// Main Function:
// The main function provides a menu-driven interface for the user to interact with the system.
// It allows the user to insert, search, delete, and display student marks:
int main() {
    HashTable hashTable(100); // Initialize hash table with size 100
    int choice;
    string studentName, subject;
    int marks;

    while (true) {
        displayMenu();
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter student name: ";
                cin >> studentName;
                cout << "Enter subject: ";
                cin >> subject;
                cout << "Enter marks: ";
                cin >> marks;
                hashTable.insertMarks(studentName, subject, marks);
                break;
            case 2:
                cout << "Enter student name: ";
                cin >> studentName;
                cout << "Enter subject: ";
                cin >> subject;
                hashTable.searchMarks(studentName, subject);
                break;
            case 3:
                cout << "Enter student name: ";
                cin >> studentName;
                cout << "Enter subject: ";
                cin >> subject;
                hashTable.deleteMarks(studentName, subject);
                break;
            case 4:
                hashTable.displayAllStudents();
                break;
            case 5:
                cout << "Exiting..." << endl;
                return 0;
            default:
                cout << "Invalid choice. Please try again." << endl;
        }
    }
    return 0;
}
```

Conclusion

The choice of a hash table for the Student Marks Management System is driven by its efficiency, dynamic size management, effective collision handling, and optimal memory utilization. Compared to other data structures, hash tables offer the best performance for the operations required by the system, ensuring quick and reliable access to student records. This design not only meets the current needs but also provides the flexibility to adapt to future requirements as the system scales.